

Smart Teams

Complete API Reference

Every core-module and directory API in the platform, documented endpoint by endpoint: what it does, who can call it, what it expects, and what it returns.

Generated July 21, 2026 • 178 endpoints across 24 modules

Intended audience: a developer integrating an external system against this API. Each module is self-contained -- read only the section(s) you need.

How To Use This Document

All endpoints live under `/api/*`. Every path is also available prefixed with `/api/v1/*` (a transparent rewrite to the exact same handler) as a stable, version-pinned contract for external integrators -- prefer `/api/v1/*` for anything you build against long-term.

AUTHENTICATION: every endpoint except the public ones explicitly marked "None" requires an Authorization: Bearer header. The token is either: 1. A user JWT from `POST /api/auth/login` (24h expiry), or 2. A Service Account API key from `POST /api/tenant/service-accounts` (does not expire until revoked) -- this is the recommended path for external/colleague integrations, since it can be scoped to exactly the privileges needed and never ties up a human login session.

PRIVILEGES: most tenant-scoped endpoints are gated by a named privilege (shown in the "Auth" column above, e.g. "leave.approve"). The full, current list of every privilege in the system -- with human-readable labels and descriptions -- is always available live from `GET /api/tenant/feature-catalog`. `tenant_admin` and `super_admin` hold every privilege implicitly and never need individual grants.

RATE LIMITING: the public/pre-auth endpoints (login, password reset, Google SSO, the public tenancy-request form) are rate-limited per IP. All other endpoints share a generous general-purpose limiter.

ERRORS: every endpoint returns `{ error: string }` with a 4xx/5xx status on failure. 403 means an auth/privilege problem; 400 means a validation problem; 404 means the referenced record doesn't exist or doesn't belong to the caller's tenant.

WEBHOOKS vs POLLING: for anything event-driven (a new leave request, a resolved ticket, a completed payroll run), prefer subscribing via `POST /api/tenant/webhooks` over polling a GET endpoint on a timer -- 18 event types are available covering every major module.

Table of Contents

1. Authentication	11 endpoints
2. Attendance (Office Check-in)	9 endpoints
3. QR Attendance	10 endpoints
4. Breaks	4 endpoints
5. Work From Home (WFH)	10 endpoints
6. Leave Management	15 endpoints
7. Payroll	14 endpoints
8. Employee Directory & Profiles	12 endpoints
9. Bulk Hiring & User Provisioning	4 endpoints
10. Roles & Permissions	6 endpoints
11. Tickets & Disputes	4 endpoints
12. Timing Alerts	2 endpoints
13. Documents	4 endpoints
14. Notifications & Push	6 endpoints
15. Devices	2 endpoints
16. Branches, Shifts & Teams	22 endpoints
17. Holidays	6 endpoints
18. Corrections & Approvals (Attendance disputes, pre-ticket system)	6 endpoints
19. Audit Ledger	4 endpoints
20. Webhooks & Service Accounts (External Integration)	6 endpoints
21. Tenant Configuration	4 endpoints
22. Analytics	2 endpoints

23. Super Admin (Platform Owner Only)

12 endpoints

24. System & Health

3 endpoints

1. Authentication

Login, password reset, Google SSO, and WebAuthn (passkey/biometric device) registration and authentication. All tokens are JWTs (24h expiry) sent as `Authorization: Bearer`.

POST /api/auth/login

AUTH: None (rate-limited)

Email + password login for any role. Returns a JWT and the user profile. If the account has mustChangePassword set (first login after being hired), the response flags this so the frontend can force a password reset before proceeding.

REQUEST

email, password

RESPONSE

{ token, user }

POST /api/auth/reset-password

AUTH: None (rate-limited)

Sets a new permanent password, either using a temp password issued at hire or a password-reset token from the forgot-password flow.

REQUEST

email, newPassword, tempPassword or resetToken

RESPONSE

{ success }

POST /api/auth/forgot-password

AUTH: None (rate-limited)

Triggers a password-reset email with a one-time token link. Always returns success even if the email doesn't exist, to avoid leaking which emails are registered.

REQUEST

email

RESPONSE

{ success }

POST /api/auth/forgot-password/confirm

AUTH: None (rate-limited)

Completes the reset using the token emailed by the forgot-password endpoint.

REQUEST

token, newPassword

RESPONSE

{ success }

POST **/api/auth/google**

AUTH: None (rate-limited)

Google OAuth sign-in for an EXISTING account only (no auto-provisioning of new users). The Google-verified email must already match a registered account.

REQUEST

credential (Google ID token)

RESPONSE

{ token, user }

POST **/api/auth/logout**

AUTH: Authenticated

Invalidates the caller's active session (single-session-per-account enforcement).

RESPONSE

{ success }

POST **/api/webauthn/register/options**

AUTH: Authenticated

Step 1 of registering a device (passkey / Windows Hello / Touch ID / Android biometric). Returns a WebAuthn challenge for navigator.credentials.create().

RESPONSE

PublicKeyCredentialCreationOptions

POST **/api/webauthn/register/verify**

AUTH: Authenticated

Step 2 of device registration — verifies the browser's attestation response and stores the public key. Marks the account's device-registration/KYC-equivalent flag complete.

REQUEST

response (attestation from navigator.credentials.create())

RESPONSE

{ success }

POST **/api/webauthn/authenticate/options**

AUTH: Authenticated

Step 1 of the daily identity check at attendance time — returns a challenge for navigator.credentials.get(). Requires device registration to already be complete.

RESPONSE

PublicKeyCredentialRequestOptions

POST**/api/webauthn/authenticate/verify****AUTH: Authenticated**

Step 2 — verifies the signed assertion and mints a short-lived 'identity pass' token that the attendance check-in endpoint (POST /api/attendance) requires as proof of live device verification.

REQUEST

response (signed assertion)

RESPONSE

```
{ identityPassToken }
```

GET**/api/webauthn/credentials****AUTH: Authenticated**

Lists the caller's own registered device credentials (self-service, no admin visibility into this).

RESPONSE

```
{ credentials: [...] }
```

2. Attendance (Office Check-in)

Core office check-in/out flow: GPS geofence + WiFi + WebAuthn identity pass verification, plus live location heartbeat while checked in and administrative correction/editing.

GET **/api/attendance/today**

AUTH: Authenticated

The caller's own check-in/out status for today, used to drive the 'mark attendance' UI state (not yet checked in / checked in / already completed).

RESPONSE

```
{ status, log }
```

GET **/api/attendance/percentage**

AUTH: Authenticated

The caller's own attendance percentage for the current month (approved check-ins vs working days so far, excluding weekends/holidays).

RESPONSE

```
{ percentage, daysPresent, workingDaysSoFar }
```

GET **/api/attendance/mine**

AUTH: Authenticated

The caller's own attendance log history within a date range.

REQUEST

startDate, endDate (query, optional)

RESPONSE

```
{ logs: [...] }
```

POST **/api/attendance**

AUTH: Authenticated

THE core check-in endpoint. Validates GPS geofence radius, optional WiFi/IP match, and the WebAuthn identity-pass token from /api/webauthn/authenticate/verify. Also handles WFH and QR-linked check-ins via attendanceMode. Late arrivals become status='pending' and route to whoever holds attendance.approve.late_arrival for approval.

REQUEST

lat, lng, identityPassToken, attendanceMode ('office'|'wfh'|'qr'), clientTimestamp, deviceId

RESPONSE

```
{ success, log, pendingApproval }
```

POST **/api/attendance/checkout**

AUTH: Authenticated

Ends the caller's current work session, computing workedMinutes (minus break time).

REQUEST

lat, lng

RESPONSE

```
{ success, log }
```

POST /api/attendance/verify-location

AUTH: Authenticated

Pre-flight GPS geofence check the frontend calls before showing the check-in button, to give an early error rather than letting the user try to submit.

REQUEST

lat, lng

RESPONSE

```
{ withinRadius, distanceMeters }
```

POST /api/attendance/verify-network

AUTH: Authenticated

Pre-flight WiFi/IP check, same purpose as verify-location.

REQUEST

simulatedIp (dev/testing only)

RESPONSE

```
{ matches }
```

POST /api/attendance/heartbeat

AUTH: Authenticated

Called periodically while checked in. Updates the user's last-known GPS fix (used by the 23:59 auto-checkout job to guess whether someone forgot to check out but is still on-site) and detects GPS drift outside the geofence DURING working hours (not on break) — raises a 'geofence_exit_working_hours' alert, routed through the escalation chain, rate-limited to one alert per 30 minutes.

REQUEST

lat, lng, simulatedIp, deviceId

RESPONSE

```
{ success, status, message? }
```

PATCH /api/tenant/attendance/:userId/:date

AUTH: attendance.edit privilege

Directly correct an already-finalized day. 'Absent' in this app is a computed state (no check-in log exists), so marking a day present CREATES the missing check-in/out rows; marking it absent REJECTS the existing check-in row (kept for audit trail, never deleted). Reflects immediately in payroll and attendance-% since both read attendance logs live. Requires a reason.

REQUEST

newStatus ('present'|'absent'), checkInTime, checkOutTime, reason

RESPONSE

```
{ success, logId }
```

3. QR Attendance

Rotating-QR-code check-in as an alternative to manual GPS check-in — a supervisor displays a QR code that rotates every N seconds, employees scan it to check in.

POST /api/qr/session/start

AUTH: attendance.qr.generate privilege

Starts a rotating QR display session for the tenant (one active session at a time — starting again while one is active returns the existing session/token instead of creating a duplicate). Requires the tenant's qrEnabled toggle + platform 'qr_attendance' feature.

RESPONSE

```
{ session, token }
```

POST /api/qr/session/stop

AUTH: attendance.qr.close privilege

Ends the active QR session.

REQUEST

sessionId

RESPONSE

```
{ success }
```

GET /api/qr/current

AUTH: attendance.qr.display or .generate privilege

Returns the current rotating token/QR image data plus live scan counts, for the display screen.

RESPONSE

```
{ session, token, scansCount, successCount, failCount, pendingCount }
```

POST /api/qr/validate

AUTH: Authenticated (employee scanning)

Employee scans a QR code — validates the token, geofence, WiFi, device-trust as configured, and issues a scan-pass token consumed by the mark-from-qr endpoint.

REQUEST

token, lat, lng, deviceId

RESPONSE

```
{ scanPassToken } or failure reason
```

POST /api/attendance/mark-from-qr

AUTH: Authenticated

Finalizes a QR-based check-in using the scan-pass token from /api/qr/validate — same downstream effect as a normal check-in (late arrival still routes to approval if needed).

REQUEST

scanPassToken, clientTimestamp

RESPONSE

```
{ success, log, pendingApproval }
```

POST **/api/qr/scans/:id/override**

AUTH: **attendance.qr.override privilege**

Manually approves a QR scan that failed automatic verification (e.g. borderline geofence).

RESPONSE

```
{ success }
```

GET **/api/qr/history**

AUTH: **attendance.qr.view_logs privilege**

Past QR session history for the tenant.

RESPONSE

```
{ sessions: [...] }
```

GET **/api/qr/logs**

AUTH: **attendance.qr.view_logs privilege**

Per-scan results (success/fail/pending) across sessions.

REQUEST

sessionId (optional filter)

RESPONSE

```
{ scans: [...] }
```

GET **/api/qr/config**

AUTH: **Authenticated**

Current QR policy for the tenant (rotation interval, GPS/WiFi/face/device-trust requirements).

RESPONSE

```
{ policy }
```

PUT **/api/qr/config**

AUTH: **tenant.config.manage privilege**

Updates the QR policy. Turning qrEnabled on requires the platform 'qr_attendance' feature to be allowed for this tenant.

REQUEST

qrEnabled, qrRotationSeconds, qrRequireGps, qrRequireWifi, qrRequireFace,
qrGeofenceRadiusMeters, qrRequireDeviceTrust

RESPONSE

```
{ success }
```

4. Breaks

Break session tracking against the tenant's daily break-time budget.

GET **/api/breaks/active**

AUTH: Authenticated

The caller's currently active (in-progress) break session, if any.

RESPONSE

```
{ activeBreak }
```

GET **/api/breaks/today**

AUTH: Authenticated

All of the caller's break sessions today, plus total minutes used against the daily budget.

RESPONSE

```
{ breaks: [...], totalMinutes }
```

POST **/api/breaks/start**

AUTH: Authenticated

Starts a break (self-service, no privilege — every clock-in-capable role can take breaks).

REQUEST

lat, lng

RESPONSE

```
{ success, breakSession }
```

POST **/api/breaks/end**

AUTH: Authenticated

Ends the active break. If ending from outside the office geofence, the break is left active (not closed) and a 'break_outside_geofence' alert is raised and routed. If the elapsed time exceeds the daily budget, a 'break_exceeded' alert is raised instead, also routed through escalation.

REQUEST

lat, lng

RESPONSE

```
{ success } or { outsideGeofence: true }
```

5. Work From Home (WFH)

An alternate attendance mode for eligible roles: register a home location once, then check in from there (subject to the same identity/network rules, with distance measured from home instead of the office).

GET **/api/attendance/wfh/eligibility**

AUTH: Authenticated

Whether the caller can use WFH today — checks tenant wfhEnabled, role allow-list, weekday allow-list, monthly quota, and device registration.

RESPONSE

```
{ eligible, reason?, needsHomeRegistration }
```

GET **/api/attendance/wfh/home-location**

AUTH: Authenticated

The caller's currently registered home location, if any.

RESPONSE

```
{ homeLocation }
```

POST **/api/attendance/wfh/register-home**

AUTH: Authenticated

One-time home location registration (self-service). Once set, it can only be CHANGED via the location-change-request approval flow below, not self-edited directly.

REQUEST

lat, lng, address

RESPONSE

```
{ success }
```

POST **/api/attendance/wfh/location-change-request**

AUTH: Authenticated

Requests changing an already-registered home location — requires approval (can't self-edit once set).

REQUEST

lat, lng, reason

RESPONSE

```
{ success, request }
```

GET **/api/attendance/wfh/location-change-requests/mine**

AUTH: Authenticated

The caller's own home-location change request history.

RESPONSE

```
{ requests: [...] }
```

GET **/api/tenant/wfh/location-change-requests**

AUTH: `attendance.approve.wfh` privilege

Pending home-location change requests awaiting approval.

RESPONSE

```
{ requests: [...] }
```

POST **/api/tenant/wfh/location-change-requests/action**

AUTH: `attendance.approve.wfh` privilege

Approve or reject a home-location change request.

REQUEST

```
requestId, action ('approve'|'reject')
```

RESPONSE

```
{ success }
```

GET **/api/tenant/wfh/stats**

AUTH: `wfh.view_logs` or `reports.view` privilege

Tenant-wide WFH usage stats (who, how often, this month).

RESPONSE

```
{ stats }
```

GET **/api/tenant/wfh/ledger**

AUTH: `wfh.view_logs` privilege

Detailed WFH check-in ledger — every WFH day, by whom, from where.

RESPONSE

```
{ ledger: [...] }
```

POST **/api/tenant/users/:id/wfh-access**

AUTH: `employee.create` privilege

Grants a specific already-hired employee individual WFH-related permissions (e.g. `wfh.view_logs`) beyond their role defaults — bounded by 'precedence of power' (granter must already hold what they're granting).

REQUEST

```
permissions: string[]
```

RESPONSE

```
{ success }
```

6. Leave Management

Leave policies (Casual/Sick/Earned/LOP/etc, each independently configured), leave requests, balance tracking, encashment, and administrative adjustments/amendments.

GET **/api/leave/mine**

AUTH: Authenticated

The caller's own leave requests and current balances per policy.

RESPONSE

```
{ requests, balances }
```

GET **/api/tenant/employees/:id/leave-balance**

AUTH: Self, or employee.read/employee.edit/reports.view

A specific employee's leave balance by policy (used by EmployeeDetailPanel).

RESPONSE

```
{ balances, remainingDays }
```

POST **/api/leave/requests**

AUTH: Authenticated

Submit a new leave request. totalDays is computed server-side from date range + half-day flag. Auto-approves immediately if the policy's requiresApproval is false, otherwise goes to whoever holds leave.approve.

REQUEST

```
policyId, leaveType, startDate, endDate, halfDay, reason, medicalCause
```

RESPONSE

```
{ success, request }
```

GET **/api/tenant/leave/policies**

AUTH: leave.read privilege

All leave policies configured for the tenant (name, code, days/year, deduction %, half-day/accrual/carry-forward/encashment rules).

RESPONSE

```
{ policies: [...] }
```

POST **/api/tenant/leave/policies**

AUTH: leave.approve privilege

Creates a new leave policy.

REQUEST

```
name, code, maxDaysPerYear, defaultDeductionPercent (0=fully paid,100=fully unpaid),  
allowHalfDay, requiresApproval, accrualEnabled, carryForwardEnabled, maxCarryForwardDays,  
encashmentEnabled
```

RESPONSE

```
{ success, policy }
```

POST /api/tenant/leave/policies/seed-defaults

AUTH: leave.approve privilege

One-click creates a starter set of common policies (Casual, Sick, Earned, Paternity, LOP, Sabbatical) for a tenant with none configured yet.

RESPONSE

```
{ success, created: [...] }
```

GET /api/tenant/leave/requests

AUTH: leave.read or leave.approve privilege

All leave requests for the tenant (approval queue), filterable by status.

REQUEST

status (optional query)

RESPONSE

```
{ requests: [...] }
```

POST /api/tenant/leave/requests/action

AUTH: leave.approve privilege

Approve or reject one pending leave request.

REQUEST

requestId, action ('approve'|'reject'), comment

RESPONSE

```
{ success, request }
```

POST /api/tenant/leave/requests/bulk-action

AUTH: leave.approve privilege

Approve or reject up to 200 pending requests in one call — same effect as the single action endpoint per row, reporting per-row success/failure rather than failing the whole batch.

REQUEST

requestIds: number[], action, comment

RESPONSE

```
{ results, updated, failed }
```

PATCH /api/tenant/leave/requests/:id/amend

AUTH: leave.edit privilege

Amends an ALREADY-DECIDED leave request's dates/type/outcome in place (distinct from .approve, which only covers the initial pending decision) — recomputes totalDays, reflects immediately in payroll. Requires a reason, appended to the request's permanent review history.

REQUEST

newStatus, startDate, endDate, leaveType, reason

RESPONSE

```
{ success, request }
```

GET /api/tenant/leave/adjustments

AUTH: leave.read or leave.approve privilege

Manual leave-balance adjustments made outside the normal request flow (e.g. correcting an accrual error).

RESPONSE

```
{ adjustments: [...] }
```

POST /api/tenant/leave/adjustments

AUTH: leave.approve privilege

Manually adds or deducts leave days for an employee outside the normal request flow.

REQUEST

userId, leaveType, days, reason

RESPONSE

```
{ success }
```

POST /api/leave/encashment

AUTH: Authenticated

Employee requests converting unused leave days into pay — only available for policies with encashmentEnabled.

REQUEST

policyId, days

RESPONSE

```
{ success, request }
```

GET /api/tenant/leave/encashment-requests

AUTH: leave.approve privilege

Pending leave-encashment requests.

RESPONSE

```
{ requests: [...] }
```

POST /api/tenant/leave/encashment-requests/action

AUTH: leave.approve privilege

Approve or reject an encashment request — the payout amount is snapshotted at approval time using the employee's effective daily rate.

REQUEST

requestId, action, comment

RESPONSE

```
{ success }
```

7. Payroll

Monthly payroll calculation (CTC, salary components, leave deductions, overtime, and optional statutory deductions — PF/ESI/Professional Tax/TDS), plus self-service payslip history and role-level compensation templates.

GET **/api/payroll/mine**

AUTH: Authenticated

The caller's own current-month payroll summary — gross, deductions, leave impact, overtime, statutory deductions if enabled, and final net pay. THE single source-of-truth calculation (buildPayrollSummary), reused everywhere else payroll numbers appear.

REQUEST

year, month (optional, default current)

RESPONSE

```
{ summary }
```

GET **/api/payroll/history**

AUTH: Authenticated

The caller's own past monthly payroll runs (payslips) — each run is snapshotted the FIRST time that month is viewed, never recalculated afterward, so a later salary change never rewrites history.

RESPONSE

```
{ runs: [...] }
```

GET **/api/payroll/history/:runId/pdf**

AUTH: Authenticated (own run only)

Downloads a specific payslip as a PDF.

RESPONSE

PDF file

GET **/api/tenant/payroll/settings**

AUTH: payroll.read privilege

Tenant-wide payroll configuration: working days/month, leave-deduction rules, overtime rate, optional-holiday limit, and the full statutory-compliance block (PF/ESI/Professional Tax/TDS toggles and rates).

RESPONSE

```
{ settings }
```

POST **/api/tenant/payroll/settings****AUTH:** payroll.manage privilege

Updates payroll settings, including every statutory-compliance field.

REQUEST

workingDaysPerMonth, maxPaidLeaveDaysPerMonth, excessLeavePenaltyPercent, overtimeHourlyRate, optionalHolidayLimit, statutoryComplianceEnabled, pfEnabled/pfEmployeeRatePercent/pfEmployerRatePercent/pfWageCeiling, esiEnabled/esiEmployeeRatePercent/esiEmployerRatePercent/esiWageCeiling, professionalTaxEnabled/professionalTaxSlabs, tdsEnabled/incomeTaxSlabs/tdsStandardDeduction, statutoryBasicPercentOfGross

RESPONSE

```
{ success, settings }
```

POST **/api/tenant/payroll/employee/:userId****AUTH:** payroll.manage privilege

Sets or updates one employee's individual compensation profile (annual CTC + named salary components: Basic/HRA/etc, each earning/deduction/employer_contribution). Snapshots the previous state into compensation history before overwriting.

REQUEST

annualCtc, overtimeHourlyRate, components: [{componentName, componentType, calculationType, value}]

RESPONSE

```
{ success, profile }
```

GET **/api/tenant/payroll/employee/:userId/history****AUTH:** payroll.read privilege

Compensation change history for one employee (what changed, when, by whom).

RESPONSE

```
{ history: [...] }
```

GET **/api/payroll/compensation-history/mine****AUTH:** Authenticated

The caller's own compensation change history.

RESPONSE

```
{ history: [...] }
```

GET **/api/tenant/payroll/employee/:userId****AUTH:** payroll.read privilege, or self

One employee's full payroll detail for a given month — attendance rows, leave rows, and the computed summary. Backs EmployeeDetailPanel's payroll section.

REQUEST

year, month

RESPONSE

```
{ attendanceRows, leaveRows, summary }
```

GET **/api/tenant/payroll/overview****AUTH:** payroll.read or reports.view privilege

Tenant-wide payroll cost overview — every employee's CTC/gross/net, aggregatable by role/department, sourced from individual profiles or role-level defaults where no individual override exists.

RESPONSE

```
{ employees: [...] }
```

GET **/api/tenant/payroll/role-defaults****AUTH:** payroll.read privilege

Template compensation packages defined per role name, applied automatically to anyone hired into that role with no individual override.

RESPONSE

```
{ roleDefaults: [...] }
```

POST **/api/tenant/payroll/role-defaults/:roleName****AUTH:** payroll.manage privilege

Sets or updates the default compensation template for a role.

REQUEST

```
annualCtc, components: [...]
```

RESPONSE

```
{ success }
```

DELETE **/api/tenant/payroll/role-defaults/:roleName****AUTH:** payroll.manage privilege

Removes a role's default compensation template.

RESPONSE

```
{ success }
```

GET **/api/earnings/mine****AUTH:** Authenticated

Day-by-day earnings breakdown for the current month (present/absent/leave/holiday per day, overtime, excess-break deductions) — the detailed counterpart to the monthly-only /api/payroll/mine summary; both reconcile to the same total.

REQUEST

```
year, month
```

RESPONSE

```
{ days: [...], summary }
```

8. Employee Directory & Profiles

The employee roster, individual profile CRUD, department management, org chart, GDPR data rights, and termination workflow.

GET

/api/tenant/employees

AUTH: `employee.read` or `reports.view` privilege

Full employee directory for the tenant (scoped to the caller's branches if branch-scoped), enriched with branch/shift/manager names.

RESPONSE

```
{ employees: [...] }
```

GET

/api/tenant/org-chart

AUTH: `employee.read` or `reports.view` privilege

Lightweight node list ({id, name, role, designation, department, managerId}) for building the reporting-hierarchy tree — the frontend assembles the tree client-side from managerId.

RESPONSE

```
{ nodes: [...] }
```

GET

/api/tenant/employees/:id

AUTH: `employee.read` or `reports.view` privilege

One employee's full profile (backs EmployeeDetailPanel).

RESPONSE

```
{ employee }
```

PUT

/api/tenant/employees/:id

AUTH: `employee.edit` or `employee.create` privilege

Updates an employee's profile fields. Also the endpoint used to reassign shift, branch, or reporting manager (managerId — drives the org chart).

REQUEST

name, email, phone, department, designation, employmentType, managerId, branchId, shiftId, dateOfJoining, employeeStatus, role, privileges

RESPONSE

```
{ success, employee }
```

GET

/api/tenant/departments

AUTH: `employee.read` or `reports.view` privilege

Named departments configured for the tenant, each with an optional department head.

RESPONSE

```
{ departments: [...] }
```

POST /api/tenant/departments

AUTH: employee.create privilege

Creates a new department.

REQUEST

name, description, headUserId

RESPONSE

```
{ success, department }
```

GET /api/employees/my-team

AUTH: Authenticated (manager)

A manager's own direct reports (self-scoped, no privilege gate beyond being someone's manager).

RESPONSE

```
{ team: [...] }
```

GET /api/employees/me/data-export

AUTH: Authenticated

GDPR 'right to access' — downloads every piece of data the app holds about the caller as one JSON file (attendance, leave, compensation history, documents metadata, shift swaps, etc). Self-service, no privilege required.

RESPONSE

JSON file download

POST /api/tenant/employees/:id/erase-data

AUTH: gdpr.manage privilege

GDPR 'right to erasure' — scrubs direct identifiers (name/email/phone/department/designation) and deletes uploaded documents for an ALREADY-TERMINATED employee. Numeric attendance/payroll history is deliberately kept (legal retention requirement), just detached from identity.

RESPONSE

```
{ success }
```

POST /api/tenant/employees/:id/terminate

AUTH: employee.terminate privilege

Terminates an employee. tenant_admin: immediate, no approval step. Anyone else holding employee.terminate: requires a reason and queues for tenant-admin (or anyone holding employee.terminate.approve) to approve.

REQUEST

reason (required for non-tenant_admin)

RESPONSE

```
{ success, terminated: true } or { pending: true, requestId }
```

GET /api/tenant/termination-requests

AUTH: employee.terminate.approve privilege

Pending termination requests awaiting approval.

RESPONSE

```
{ requests: [...] }
```

POST

/api/tenant/termination-requests/action

AUTH: `employee.terminate.approve` privilege

Approve or reject a termination request.

REQUEST

`requestId, action ('approve'|'reject')`

RESPONSE

`{ success }`

9. Bulk Hiring & User Provisioning

Onboarding new employees, one at a time or via CSV batch, and self-service employee-facing lookups.

POST /api/tenant/users/create

AUTH: `employee.create` privilege

Hires one employee — assigns role/branch/shift, optionally grants additional privileges beyond the role's defaults (bounded by 'precedence of power': the hirer can only grant privileges they themselves hold). Auto-registers a brand-new custom role name if it doesn't exist yet. Sends a credential email with a temp password.

REQUEST

email, name, role, branchId, shiftId, department, privileges (additional), additionalBranchIds

RESPONSE

```
{ success, isNewRole, emailDelivered }
```

POST /api/tenant/users/bulk-create

AUTH: `employee.create` privilege

Hires up to 200 employees from one CSV-derived batch. Each row gets its role's default privileges only (no per-row custom privilege grants). One bad row doesn't fail the batch — every row reports its own success/failure.

REQUEST

rows: [{email, name, role, branchId, shiftId, department}]

RESPONSE

```
{ results, created, failed }
```

GET /api/tenant/users

AUTH: `employee.read` or `employee.create` privilege

Simple user list for the tenant (lighter-weight than the full directory endpoint).

RESPONSE

```
{ users: [...] }
```

POST /api/tenant/users/:id/qr-access

AUTH: `employee.create` privilege

Grants a specific already-hired employee individual QR-related permissions (generate/display/close/override/view_logs) beyond their role defaults.

REQUEST

permissions: string[]

RESPONSE

```
{ success }
```

10. Roles & Permissions

The feature-toggle catalog — every delegable capability in the app, custom role definitions, and the 'what can I grant' precedence-of-power check.

GET **/api/tenant/feature-catalog**

AUTH: Authenticated

The full server-driven catalog of every toggleable feature/privilege in the app, grouped by category, plus a dependency map (e.g. a 'resolve' toggle requires its matching 'receive' toggle). Powers the Role Permissions editor and the hire form's additional-access grid.

RESPONSE

```
{ catalog, dependencies }
```

GET **/api/tenant/my-privileges**

AUTH: Authenticated

The caller's own effective privilege list ('ALL' for tenant_admin/super_admin, else an array) — lets the frontend know exactly which toggles it's allowed to grant to someone else.

RESPONSE

```
{ privileges }
```

GET **/api/tenant/my-branches**

AUTH: Authenticated

Branches the caller can manage (all of them if unrestricted, else their assigned branch + any branch.multi_access grants).

RESPONSE

```
{ branches: [...] }
```

GET **/api/tenant/roles**

AUTH: Authenticated

Every role name defined for the tenant and its default privilege set.

RESPONSE

```
{ roles: [...] }
```

POST **/api/tenant/roles**

AUTH: roles.manage privilege

Defines a brand-new custom role name with a chosen privilege bundle — bounded by precedence of power. Also requires the platform 'custom_rbac' feature to be allowed for the tenant.

REQUEST

```
roleName, privileges: string[]
```

RESPONSE

```
{ success, role }
```

PATCH**/api/tenant/roles/:id****AUTH:** roles.manage privilege

Edits an existing role's default privilege set — takes effect immediately for everyone currently in that role (privileges are resolved live, not frozen at hire time). Only newly-ADDED privileges are checked against the editor's own holdings; removals always pass through.

REQUEST

privileges: string[]

RESPONSE

{ success }

11. Tickets & Disputes

General-purpose employee dispute/request system with priority levels and an automatic manager -> GM -> tenant_admin escalation chain (24-hour auto-forward on inaction).

POST /api/tickets

AUTH: Authenticated

Raises a ticket (attendance_dispute / leave_dispute / payroll_dispute / other) with a priority the raiser sets themselves. Auto-routes to the raiser's manager (via managerId or team), falling back to the tenant's GM, then tenant_admin if neither exists.

REQUEST

category, priority, subject, description, relatedDate (optional, for attendance disputes), relatedAttendanceLogId, relatedLeaveRequestId

RESPONSE

```
{ ticket }
```

GET /api/tickets/mine

AUTH: Authenticated

The caller's own raised tickets and their status.

RESPONSE

```
{ tickets: [...] }
```

GET /api/tenant/tickets

AUTH: Authenticated (routing-based)

Tickets currently assigned to the caller, plus — for anyone holding tickets.manage — every ticket tenant-wide (an oversight override, not a routing change).

RESPONSE

```
{ tickets: [...] }
```

POST /api/tenant/tickets/:id/action

AUTH: Must be current assignee, or hold tickets.manage

Resolve, reject, or escalate a ticket. Resolving an attendance_dispute or leave_dispute can include the actual correction (attendanceEdit/leaveEdit) in the same call — applied via the exact same logic the standalone edit endpoints use, but STILL requires the resolver to separately hold attendance.edit/leave.edit (being the assignee alone doesn't grant that).

REQUEST

action ('resolve'|'reject'|'escalate'), resolutionNote, attendanceEdit: {newStatus, checkInTime, checkOutTime}, leaveEdit: {newStatus, startDate, endDate, leaveType}

RESPONSE

```
{ success, ticket }
```

12. Timing Alerts

Break-overstay, GPS-out-of-bounds, security/fraud, and low-attendance alerts. Routed through the same escalation chain as tickets; per-type receive/resolve toggles gate visibility and action beyond one's own routed queue.

GET **/api/tenant/alerts**

AUTH: Routing-based + optional per-type '.receive' privileges

Alerts visible to the caller — always includes anything currently routed to them, plus (for anyone holding the matching per-type or general '.receive' privilege) every alert of that type tenant-wide.

RESPONSE

```
{ alerts: [...] }
```

POST **/api/tenant/alerts/action**

AUTH: Routing-based + optional per-type '.resolve' privileges

Accept or reject an alert — same routed-to-you-OR-privileged split as viewing.

REQUEST

```
alertId, action ('accept'|'reject')
```

RESPONSE

```
{ success }
```

13. Documents

Employee document storage (offer letters, contracts, ID proof, certificates) — a tenant-wide opt-in module.

POST /api/tenant/documents

AUTH: Self, or employee.read/employee.edit/employee.create/reports.view

Uploads a document for an employee (base64 in JSON, 15MB limit). Only works if the tenant's documentsEnabled toggle and the platform 'documents' feature are both on.

REQUEST

userId, category, fileName, mimeType, fileBase64

RESPONSE

```
{ success, document }
```

GET /api/tenant/documents

AUTH: Self, or employee.read/employee.edit/reports.view

Lists documents for an employee.

REQUEST

userId

RESPONSE

```
{ documents: [...] }
```

GET /api/tenant/documents/:id/download

AUTH: Self, or employee.read/employee.edit/reports.view

Downloads a specific document's file content.

RESPONSE

File download

DELETE /api/tenant/documents/:id

AUTH: Self, or employee.create/employee.edit

Deletes a document (deletion requires the stricter privilege set — the owner or someone who can actually manage the roster, not merely read/report access).

RESPONSE

```
{ success }
```

14. Notifications & Push

In-app notifications and browser/PWA push notifications (Web Push / VAPID) — every in-app notification automatically also attempts a push to any subscribed device.

GET **/api/tenant/notifications**

AUTH: Authenticated

The caller's own in-app notifications.

RESPONSE

```
{ notifications: [...] }
```

POST **/api/tenant/notifications/:id/read**

AUTH: Authenticated

Marks one notification read.

RESPONSE

```
{ success }
```

POST **/api/tenant/notifications/read-all**

AUTH: Authenticated

Marks all of the caller's notifications read.

RESPONSE

```
{ success }
```

GET **/api/push/vapid-public-key**

AUTH: Authenticated

Returns the public VAPID key the frontend needs to create a Push subscription, and whether push is configured server-side at all.

RESPONSE

```
{ publicKey, configured }
```

POST **/api/push/subscribe**

AUTH: Authenticated

Registers a browser push subscription for the caller (self-service — every user does this for their own device, like WebAuthn registration).

REQUEST

```
endpoint, keys: {p256dh, auth}
```

RESPONSE

```
{ success }
```

POST

/api/push/unsubscribe

AUTH: Authenticated

Removes a push subscription.

REQUEST

endpoint

RESPONSE

{ success }

15. Devices

Approving an employee's request to switch their registered device (re-register WebAuthn on a new phone/laptop).

GET

/api/tenant/device-requests

AUTH: `settings.edit privilege`

Pending device-change requests.

RESPONSE

```
{ requests: [...] }
```

POST

/api/tenant/device-requests/action

AUTH: `settings.edit privilege`

Approve or reject a device-change request.

REQUEST

`requestId, action`

RESPONSE

```
{ success }
```

16. Branches, Shifts & Teams

Physical branch/outlet setup (location, geofence radius, policies), named shifts per branch, temporary per-employee shift overrides, two-party shift swaps, and manager-owned teams.

GET

/api/branches

AUTH: Authenticated

Branches visible to the caller (scoped if branch-restricted).

RESPONSE

```
{ branches: [...] }
```

GET

/api/branches/:id

AUTH: Authenticated

One branch's full detail.

RESPONSE

```
{ branch }
```

POST

/api/branches/bulk

AUTH: branch.manage privilege

First-login branch-setup wizard — creates multiple branches (each with a default shift) in one call, and marks the tenant's branchSetupCompleted flag.

REQUEST

```
branches: [{name, lat, lng, radius, shiftStart, shiftEnd, ...}]
```

RESPONSE

```
{ success, branches: [...] }
```

POST

/api/branches

AUTH: branch.manage privilege

Creates one new branch.

REQUEST

```
name, lat, lng, radiusMeters, ...
```

RESPONSE

```
{ success, branch }
```

PATCH

/api/branches/:id

AUTH: branch.manage privilege

Updates a branch's location/radius/policies.

REQUEST

```
any branch field
```

RESPONSE

```
{ success }
```

POST /api/geocode/forward

AUTH: Authenticated

Converts a free-text address into lat/lng (used by the branch/home-location setup forms). Uses free OpenStreetMap Nominatim.

REQUEST

address

RESPONSE

```
{ lat, lng }
```

GET /api/branches/:branchId/shifts

AUTH: Authenticated

Named shifts defined for a branch.

RESPONSE

```
{ shifts: [...] }
```

POST /api/branches/:branchId/shifts

AUTH: shift.manage privilege

Defines a new named shift for a branch.

REQUEST

name, checkInTime, checkOutTime, isDefault

RESPONSE

```
{ success, shift }
```

PATCH /api/shifts/:id

AUTH: shift.manage privilege

Updates a shift's timing.

REQUEST

checkInTime, checkOutTime, name

RESPONSE

```
{ success }
```

POST /api/tenant/employees/:id/shift-override

AUTH: shift.manage privilege

Applies a dated temporary shift change for one employee, without touching their permanent shift assignment.

REQUEST

shiftId, startDate, endDate, reason

RESPONSE

```
{ success, override }
```

GET /api/tenant/employees/:id/shift-overrides

AUTH: shift.manage privilege, or self

An employee's temporary shift-change history.

RESPONSE

```
{ overrides: [...] }
```

DELETE /api/tenant/employees/:id/shift-overrides/:overrideId

AUTH: shift.manage privilege

Cancels a temporary shift change.

RESPONSE

```
{ success }
```

POST /api/tenant/shift-swap

AUTH: Authenticated

Requests swapping the caller's shift on a specific date with a named colleague — step 1 of a 3-step flow (request -> target accepts -> shift.manage holder approves).

REQUEST

targetUserId, swapDate, reason

RESPONSE

```
{ success, request }
```

GET /api/tenant/shift-swap/mine

AUTH: Authenticated

The caller's own shift-swap requests (as requester or target).

RESPONSE

```
{ requests: [...] }
```

POST /api/tenant/shift-swap/:id/respond

AUTH: Authenticated (must be the named target)

The target colleague accepts or declines the swap proposal.

REQUEST

accept: boolean

RESPONSE

```
{ success }
```

GET /api/tenant/shift-swap/pending-approval

AUTH: shift.manage privilege

Swaps both parties agreed to, awaiting final policy sign-off.

RESPONSE

```
{ requests: [...] }
```

POST /api/tenant/shift-swap/:id/action

AUTH: shift.manage privilege

Final approve/reject of an already-mutually-agreed swap.

REQUEST

action ('approve' | 'reject')

RESPONSE

```
{ success }
```

GET **/api/tenant/teams/mine**

AUTH: Authenticated (manager)

The caller's own team (if they've created one) and its members.

RESPONSE

```
{ team, members }
```

POST **/api/tenant/teams**

AUTH: team.manage privilege (never tenant_admin — deliberately excluded)

Creates a manager's own team. One team per manager by design.

REQUEST

name

RESPONSE

```
{ success, team }
```

GET **/api/tenant/teams/candidates**

AUTH: team.manage privilege

Colleagues in the same department not yet on the caller's team, eligible to be added.

RESPONSE

```
{ candidates: [...] }
```

POST **/api/tenant/teams/members**

AUTH: team.manage privilege

Adds a colleague to the caller's team.

REQUEST

userId

RESPONSE

```
{ success }
```

DELETE **/api/tenant/teams/members/:userId**

AUTH: team.manage privilege

Removes a member from the caller's team.

RESPONSE

```
{ success }
```

17. Holidays

Company holiday calendar (fixed, admin-maintained) plus employee-selectable optional/floating holidays within a tenant-configured limit.

GET

/api/tenant/holidays

AUTH: Authenticated

The tenant's holiday calendar.

RESPONSE

```
{ holidays: [...] }
```

POST

/api/tenant/holidays

AUTH: holiday.manage privilege

Adds a holiday.

REQUEST

date, name

RESPONSE

```
{ success }
```

DELETE

/api/tenant/holidays/:id

AUTH: holiday.manage privilege

Removes a holiday.

RESPONSE

```
{ success }
```

GET

/api/tenant/holidays/optional

AUTH: Authenticated

The caller's optional-holiday selections and remaining quota this year.

RESPONSE

```
{ selections, remaining }
```

POST

/api/tenant/holidays/optional

AUTH: Authenticated

Selects an optional holiday (self-service, limited by the tenant's optionalHolidayLimit payroll setting).

REQUEST

date, name

RESPONSE

```
{ success }
```

POST

/api/tenant/holidays/import-public

AUTH: holiday.manage privilege

Bulk-imports a country/region's public holidays for the year from a public holiday API.

REQUEST

countryCode, regionCode, year

RESPONSE

```
{ success, imported: [...] }
```

18. Corrections & Approvals (Attendance disputes, pre-ticket system)

The original, narrower attendance-correction flow (missed punch / wrong location) that predates the general Tickets system — still active as the lighter-weight option, and the pending-late-arrival approval queue.

POST `/api/attendance/corrections`

AUTH: **Authenticated**

Employee flags a missed check-in/out or wrong-location flag for a specific date, with a reason. Does NOT auto-modify the original attendance log — creates its own resolvable record reviewed alongside it.

REQUEST

`requestType ('missed_checkin'|'missed_checkout'|'wrong_location'|'other'), requestedDate, requestedTime, reason`

RESPONSE

`{ success, correction }`

GET `/api/attendance/corrections/mine`

AUTH: **Authenticated**

The caller's own correction request history.

RESPONSE

`{ corrections: [...] }`

GET `/api/tenant/corrections`

AUTH: **attendance.approve.corrections privilege**

Pending correction requests awaiting review.

RESPONSE

`{ corrections: [...] }`

POST `/api/tenant/corrections/action`

AUTH: **attendance.approve.corrections privilege**

Approve or reject a correction request.

REQUEST

`correctionId, action`

RESPONSE

`{ success }`

GET `/api/tenant/attendance/pending`

AUTH: **attendance.approve.late_arrival privilege**

Late check-ins awaiting approval (status='pending' attendance logs).

RESPONSE

`{ logs: [...] }`

POST

/api/tenant/attendance/action

AUTH: attendance.approve.late_arrival privilege

Approve or reject a pending late check-in.

REQUEST

logId, action

RESPONSE

{ success }

19. Audit Ledger

A tamper-evident, hash-chained log of every significant action in the tenant (SHA-256 chained — each entry's hash includes the previous entry's hash, so any retroactive edit breaks the chain and is detectable).

GET **/api/audit/mine**

AUTH: **Authenticated**

The caller's own audit trail — actions they took, or that were taken about them. Self-service, no privilege required.

RESPONSE

```
{ ledger: [...] }
```

GET **/api/tenant/attendance-archive**

AUTH: **reports.view privilege**

Archived attendance logs older than the tenant's retention window (moved off the hot table by the monthly archival job, still queryable here).

RESPONSE

```
{ archived: [...] }
```

GET **/api/tenant/ledger**

AUTH: **reports.view privilege**

The full tenant-wide audit ledger.

RESPONSE

```
{ ledger: [...] }
```

POST **/api/tenant/ledger/verify**

AUTH: **reports.view privilege**

Recomputes the hash chain from genesis and confirms it matches every stored hash — proves the ledger hasn't been tampered with.

RESPONSE

```
{ valid: boolean, brokenAtId? }
```

20. Webhooks & Service Accounts (External Integration)

Outbound event webhooks for external systems to subscribe to, and scoped machine-to-machine API keys for external systems to call INTO this API. THIS IS THE PRIMARY INTEGRATION SURFACE for a colleague building against this app externally.

GET `/api/tenant/webhooks`

AUTH: `webhooks.manage privilege`

Lists configured outbound webhook subscriptions (never returns the signing secret again after creation).

RESPONSE

```
{ webhooks: [...], availableEvents: [...] }
```

POST `/api/tenant/webhooks`

AUTH: `webhooks.manage privilege`

Creates a webhook subscription. Every delivery is POSTed as JSON with an X-SmartTeams-Signature HMAC header (signed with the secret returned ONCE here) so the receiver can verify authenticity. 18 event types available: attendance.checked_in/checked_out/correction_resolved/break_violation/edited, leave.requested/approved/rejected/amended, payroll.run_completed, employee.terminated, shift.swap_approved, ticket.raised/resolved/escalated.

REQUEST

`url` (must be https in production), `events`: `string[]`

RESPONSE

```
{ webhook, signingSecret (shown once) }
```

DELETE `/api/tenant/webhooks/:id`

AUTH: `webhooks.manage privilege`

Removes a webhook subscription.

RESPONSE

```
{ success }
```

GET `/api/tenant/service-accounts`

AUTH: `serviceAccounts.manage privilege`

Lists machine-to-machine API keys (metadata only — the raw key is never retrievable after creation, only rotate/revoke).

RESPONSE

```
{ serviceAccounts: [...] }
```

POST **/api/tenant/service-accounts****AUTH:** `serviceAccounts.manage privilege`

Issues a new API key scoped to a specific privilege list (bounded by precedence of power — can only be granted privileges the issuer holds). THE recommended way for a colleague's external system to authenticate: call any endpoint above with 'Authorization: Bearer ' instead of a user JWT, and it will be authorized exactly as if a user held that key's privilege list.

REQUEST

```
name, privileges: string[]
```

RESPONSE

```
{ serviceAccount, apiKey (shown once) }
```

DELETE **/api/tenant/service-accounts/:id****AUTH:** `serviceAccounts.manage privilege`

Revokes an API key (soft delete — past usage stays in the audit history).

RESPONSE

```
{ success }
```

21. Tenant Configuration

Org-wide policy settings: geofence/network rules, shift defaults, security toggles, and the company policy announcement banner.

GET **/api/tenant/config**

AUTH: **Authenticated**

The full tenant policy row (WiFi SSID/IP, GPS geofence coordinates+radius, shift defaults, grace period, break budget, weekend config, min attendance %, WFH policy block, KYC/document/password/idle-timeout toggles).

RESPONSE

```
{ tenant }
```

POST **/api/tenant/config/update**

AUTH: **tenant.config.manage privilege**

Updates tenant policy. Turning on documentsEnabled/wfhEnabled/kycEnabled/wifiCheckEnabled requires the corresponding platform-level feature to be allowed for this tenant (a super-admin-controlled plan restriction) — turning OFF is always allowed regardless.

REQUEST

```
wifiSsid, officeIp, wifiCheckEnabled, lat, lng, radius, shiftStart, shiftEnd, gracePeriodMins,
halfDayMins, dailyBreakBudgetMins, weekendConfig, minAttendancePercent, wfhEnabled+policy
fields, kycEnabled, documentsEnabled, passwordExpiryDays, idleTimeoutMinutes,
attendanceRetentionMonths
```

RESPONSE

```
{ success }
```

GET **/api/tenant/policy**

AUTH: **Authenticated**

Just the company policy announcement text (narrow endpoint so every employee's browser doesn't have to fetch the full config row just to show a banner).

RESPONSE

```
{ policyAnnouncement, policyAnnouncementUpdatedAt }
```

PUT **/api/tenant/policy**

AUTH: **tenant.policy.manage privilege**

Updates the company policy announcement banner shown on every dashboard.

REQUEST

```
policyAnnouncement
```

RESPONSE

```
{ success }
```

22. Analytics

Tenant-wide attendance/leave/payroll dashboards and trend charts.

GET

/api/tenant/analytics

AUTH: Authenticated (breakdown requires reports.view or employee.read)

Today's snapshot: total/present/absent/late/rejected counts. The per-person breakdown (needed for drill-down lists) only included if the caller holds reports.view or employee.read — raw counts always returned regardless.

RESPONSE

```
{ totalStaff, presentToday, absentToday, lateToday, rejectedToday, breakdown? }
```

GET

/api/tenant/analytics/trends

AUTH: reports.view privilege

Historical trend data for charts (attendance %, leave usage, payroll cost over time).

REQUEST

range (e.g. days)

RESPONSE

```
{ series: [...] }
```

23. Super Admin (Platform Owner Only)

Cross-tenant platform administration — onboarding new companies, suspending/deleting tenants, and the platform-level feature-plan whitelist that bounds what any tenant admin can ever turn on. All endpoints in this module require role='super_admin'.

POST **/api/tenancy/request**

AUTH: None (public, rate-limited)

Public 'request a company account' form — the entry point for a brand-new prospective tenant.

REQUEST

companyName, adminName, adminEmail, plan

RESPONSE

{ success }

GET **/api/super/requests**

AUTH: super_admin

Pending tenancy signup requests.

RESPONSE

{ requests: [...] }

GET **/api/super/notifications**

AUTH: super_admin

Platform-level notification inbox (not tied to a specific user).

RESPONSE

{ notifications: [...] }

POST **/api/super/approve**

AUTH: super_admin

Approves a tenancy request — creates the tenant, seeds starter role defaults, and mails the new tenant_admin their credentials. featuresAllowed here sets the platform-level module whitelist for this tenant.

REQUEST

requestId, plan, featuresAllowed: string[]

RESPONSE

{ success, tenant }

GET **/api/super/platform-features**

AUTH: super_admin

The canonical list of platform-level module keys (device_identity, wifi_lock, gps_geofence, documents, qr_attendance, wfh, webhooks, service_accounts, custom_rbac) used to build featuresAllowed.

RESPONSE

{ features: [...] }

GET **/api/super/tenants****AUTH:** `super_admin`

Every tenant on the platform with employee counts and plan info.

RESPONSE

```
{ tenants: [...] }
```

POST **/api/super/tenants/status****AUTH:** `super_admin`

Suspends or reactivates a tenant — suspension immediately blocks that tenant's users from logging in or checking in.

REQUEST

```
tenantId, status ('active'|'suspended')
```

RESPONSE

```
{ success }
```

POST **/api/super/tenants/features****AUTH:** `super_admin`

Edits an EXISTING tenant's platform feature whitelist (the ongoing counterpart to the one-time selection at approval). This is the top of the whole toggle cascade — whatever isn't checked here, no tenant admin can ever turn on or delegate, regardless of their own privileges.

REQUEST

```
tenantId, featuresAllowed: string[]
```

RESPONSE

```
{ success, featuresAllowed }
```

POST **/api/super/tenants/delete****AUTH:** `super_admin`

PERMANENTLY deletes a tenant and everything belonging to it (one-way door — no soft-delete/undo). Audit-ledger entries are detached rather than deleted, preserving the hash chain.

REQUEST

```
tenantId
```

RESPONSE

```
{ success }
```

GET **/api/super/tenants/:tenantId/admins****AUTH:** `super_admin`

Lists the tenant_admin account(s) for a specific tenant.

RESPONSE

```
{ admins: [...] }
```

POST**/api/super/tenant-admins/delete****AUTH:** `super_admin`

Permanently deletes one tenant_admin login (does not affect the tenant or its data — just that one account).

REQUEST`userId`**RESPONSE**`{ success }`

GET**/api/super/analytics****AUTH:** `super_admin`

Cross-tenant platform usage analytics (plan breakdown, tenant counts, etc).

RESPONSE`{ analytics }`

24. System & Health

Unauthenticated infrastructure endpoints.

GET **/api/health**

AUTH: None (public)

Basic liveness check.

RESPONSE

```
{ status: 'ok' }
```

GET **/api/health/db**

AUTH: None (public)

Database connectivity check.

RESPONSE

```
{ status, latencyMs }
```

GET **/api/openapi.json**

AUTH: None (public)

The machine-readable OpenAPI 3.0 spec for this API (also browsable as Swagger UI at /api/docs).

RESPONSE

OpenAPI 3.0 JSON document
